

Chapitre 3 : Création de nos premières pages

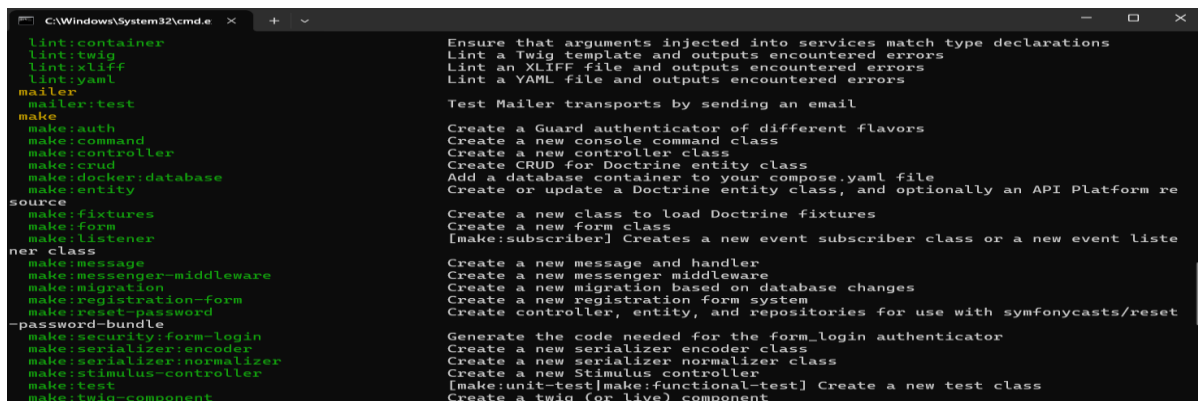
On va créer notre première page avec Symfony. Par défaut on arrivait sur la page d'accueil « **Welcome to Symfony 7** ». Pour changer ça on va devoir créer dans un premier temps un **Controller**. C'est une classe qui va contenir différentes méthodes, chaque méthode correspondra à une url particulière. On pourrait créer nous-mêmes ces classes mais Symfony permet de les créer automatiquement via la console (CMD).

3.1 La page index et les routes

Pour créer un **Controller**, il faut regarder les commandes

- **symfony console**

Cette commande vous liste toutes les commandes à votre disposition dans votre projet.



```
C:\Windows\System32\cmd.e x + v
lint:container          Ensure that arguments injected into services match type declarations
lint:twig               Lint a Twig template and outputs encountered errors
lint:xliff              Lint an XLIFF file and outputs encountered errors
lint:yaml               Lint a YAML file and outputs encountered errors
mailer                 Test Mailer transports by sending an email
make                    Create a new class
make:auth               Create a Guard authenticator of different flavors
make:command            Create a new console command class
make:controller          Create a new controller class
make:crud               Create CRUD for Doctrine entity class
make:doctrine:database  Add a database container to your compose.yaml file
make:entity             Create or update a Doctrine entity class, and optionally an API Platform re
source                 Create a new class to load Doctrine fixtures
make:fixtures           Create a new form class
make:form               [make:subscriber] Creates a new event subscriber class or a new event liste
ner class              Create a new message and handler
make:message            Create a new messenger middleware
make:migration          Create a new migration based on database changes
make:registration-form  Create a new registration form system
make:reset-password     Create controller, entity, and repositories for use with symfonycasts/reset
password-bundle        Generate the code needed for the form_login authenticator
make:security:form-login Create a new serializer encoder class
make:serializer:encoder Create a new serializer normalizer class
make:serializer:normalizer Create a new Stimulus controller
make:stimulus-controller [make:unit-test|make:functional-test] Create a new test class
make:test               Create a twig (or live) component
make:twig-component
```

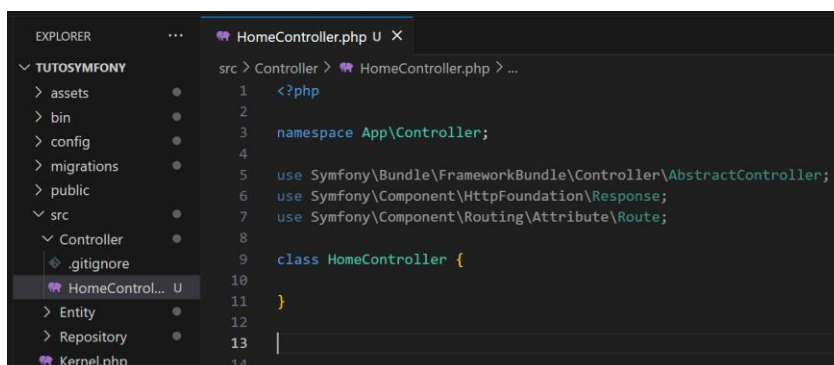
On verra qu'il y a un descriptif de chacune des commandes disponibles, on fera donc :

- **symfony console make:controller nomController**(pas obliger de noter Controller, il le rajoutera)

On va l'appeler **HomeController** ce qui donne donc **symfony console make:controller Home**.

Automatiquement il crée 2 fichiers, le **Controller HomeController.php** qui est une classe PHP dans le dossier **/src** et la vue (**View**) **index.html.twig** dans le dossier **/templates**.

Dans VScode allez là où se trouve votre Controller et ouvrez-le. Dans un premier temps on va supprimer ce qu'il y a dedans et vous aurez quelques choses comme ça :



```
EXPLORER
TUTOSYMFONY
  > assets
  > bin
  > config
  > migrations
  > public
  > src
    > Controller
    > Entity
    > Repository
    > Kernel.php
  > .gitignore
  > HomeController.php U
  > Entity
  > Repository
  > Kernel.php

src > Controller > HomeController.php > ...
1  <?php
2
3  namespace App\Controller;
4
5  use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6  use Symfony\Component\HttpFoundation\Response;
7  use Symfony\Component\Routing\Attribute\Route;
8
9  class HomeController {
10
11  }
12
13
14
```

On va repartir d'une classe simple.

Un Controller, je rappelle que c'est tout simplement une classe qui va contenir des méthodes et chaque méthode va correspondre à une page.

On va créer notre première méthode qui sera l'index sans paramètres donc notre page d'accueil et non plus la page d'accueil Symfony.

Les méthodes dans les Controllers doivent toujours renvoyer une réponse (qui est un Objet qui provient de Symfony : **Response**). Attention celle qui vient de HttpFoundation.

Donc je vais retourner une Response, qui lorsqu'on le construit, reçoit en premier paramètre le contenu, on va mettre « **Hello World !!!** ».

- **return new Response("Hello World !!!") ;**

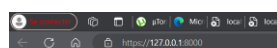
Maintenant il va falloir expliquer au Framework que l'URL racine va correspondre à cette méthode-là. Il y a deux manières de faire. Je vais vous montrer les deux bien que j'en utiliserai qu'une seule. Les deux sont importantes à savoir.

La première manière c'est d'aller dans le dossier config à la racine de votre projet, dans le fichier « **routes.yaml** ». Là on va donner le nom de notre route donc « **home** », lui donner son chemin avec « **path** » qui sera la racine donc « **/** ». On écrit ensuite « **controller** » à qui on donnera le nom de notre controller donc en précisant son namespace et la méthode qu'on veut exécuter qui est donc index « **App\Controller\HomeController::index** » :

```
config > ! routes.yaml
1 controllers:
2   resource:|
3     path: ../src/Controller/
4     namespace: App\Controller
5     type: attribute
6 home:
7   path: /
8   controller: App\Controller\HomeController::index
```

En gros j'explique à mon Framework que lorsqu'on a l'URL racine « **/** » il faut qu'il aille chercher cette action « **index** ». Attention dans YAML l'indentation est importante.

Une fois que c'est fait vous pourrez retourner dans votre navigateur et faire un refresh. Vous verrez votre Hello World !!! :



Hello World !!!

Voici la première manière de faire, qui était l'ancienne manière de faire. Beaucoup d'autres Frameworks utilisent ce système de mettre dans un fichier à part toutes les routes.

Vous verrez en Django et en React que ça se fait aussi. Le plus important c'est que vous compreniez comment ça fonctionne.

Maintenant je vais vous montrer la deuxième manière de faire qu'on fera tout le temps dans mon cours de Symfony.

Mettez en commentaire ce qu'on vient de faire dans « **routes.yaml** ». Les commentaires dans les fichiers YAML se font avec « # », vous pouvez faire dans Visual studio « Ctrl+/ ».

```
! routes.yaml M X
config > ! routes.yaml
1 controllers:
2     resource:
3         path: ../src/Controller/
4         namespace: App\Controller
5         type: attribute
6 # home:
7 #     path: /
8 #     controller: App\Controller\HomeController::index
```

La partie en haut, ça permet de dire à Symfony, tu peux scanner les Controllers et si tu vois certains attributs, tu peux enregistrer les routes automatiquement.

On peut retourner dans notre Controller. C'est là qu'on va ajouter des attributs routes, directement au-dessus de nos méthodes qu'on a créées. Cet attribut vient du Namespace :

```
- use Symfony\Component\Routing\Attribute\Route;
```

Qui était déjà présent dans notre HomeController. Le « use » c'est comme un require ou un include, ça permet d'importer. Voici la syntaxe des routes :

```
- #[Route("leCheminDeLUrl", "nomDeLaRoute")]
```

Le premier paramètre est donc le chemin de l'URL donc « / » et le deuxième paramètre c'est le nom de notre route « **home** ». On va nommer les paramètres, ça permettra de mieux les différencier.

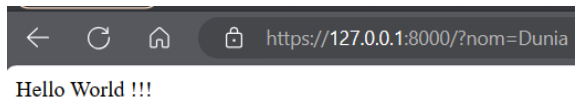
```
#[Route(path : "/", name : "home")]
public function index(): Response{
    return new Response("Hello World !!!");
}
```

Si vous retournez actualiser votre navigateur vous verrez toujours Hello World !!! Voilà la manière qu'on va utiliser à chaque fois. On a directement la route qui est lié à notre action. C'est plus pratique mais c'est bien de connaître aussi que l'autre manière existe.

```
HomeController.php U X
src > Controller > HomeController.php > HomeController
1 <?php
2
3 namespace App\Controller;
4
5 use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6 use Symfony\Component\HttpFoundation\Response;
7 use Symfony\Component\Routing\Attribute\Route;
8
9 class HomeController extends AbstractController {
10
11     #[Route(path : "/", name : "home")]
12     public function index(): Response{
13         return new Response("Hello World !!!");
14     }
15
16 }
```

Remettez aussi le « **extends AbstractController** » à côté de la classe. C'est de l'héritage, donc notre controller **HomeController** hérite de la classe **AbstractController**. Ça va donc nous fournir des méthodes génériques. Par exemple grâce à cette classe parent, on pourra faire « `$this->redirect` » qui est fournis donc dans **AbstractController**.

On a vu l'objet **Response** mais on va aussi parler d'un autre objet très important (**Request**) qui va nous permettre de gérer ce qui rentre dans le Framework. Rappelez-vous en PHP on pouvait mettre des informations dans l'URL et les récupérer. Ici on va essayer de faire ça.



Je vais mettre « **?nom=Dunia** »

En PHP dans le code on aurait utilisé une variable globale donc j'aurai mis « `$_GET['nom']` ».

```
#[Route(path : "/", name : "home")]
public function index(): Response{
    return new Response("Hello " . $_GET['nom']);
}
```

Et si vous testez vous allez voir que ça va bien afficher Hello Dunia. Ça fonctionne parce que je rappelle que Symfony c'est du PHP. Mais ce n'est pas la manière optimale à faire dans Symfony. Si maintenant vous enlevez de l'URL « **?nom=Dunia** », ça va nous faire une erreur.



Nous on va plutôt mettre dans notre méthode « **index** » un paramètre de type **Request** (**Attention** il faut importer celui de **HttpFoundation** comme Response.)

```
- use Symfony\Component\HttpFoundation\Request;
```

Normalement Visual Studio vous l'importe automatiquement quand vous le choisissez.

Ensuite si on veut récupérer des paramètres dans l'URL, on a dans cet objet « **\$request** » des méthodes utiles. Je dis je veux récupérer dans query le « **nom** » ce qui donne :

```
HomeController.php U X
src > Controller > HomeController.php > ...
1  <?php
2
3  namespace App\Controller;
4
5  use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6  use Symfony\Component\HttpFoundation\Request;
7  use Symfony\Component\HttpFoundation\Response;
8  use Symfony\Component\Routing\Attribute\Route;
9
10 class HomeController extends AbstractController {
11
12     #[Route(path : "/", name : "home")]
13     public function index(Request $request): Response{
14         return new Response("Hello " . $request->query->get('nom'));
15     }
16
17 }
```

Si vous testez vous verrez que ça bien permis de récupérer la valeur de « **nom** » dans votre URL. Si on enlève la suite dans l'URL, on aura plus l'erreur qu'on avait eu en utilisant basiquement la variable globale, il dira juste hello. On peut rajouter une valeur par défaut dans le **get()**.

```
- $request->query->get('nom', 'World !!!')
```

Si vous n'avez rien dans l'URL il mettra Hello World !!! sinon si on met ?nom=Deo il mettra Hello Deo.

Il faut savoir qu'on peut comme en PHP utiliser un **var_dump()** pour savoir ce qu'il y a dans une variable. A savoir qu'on peut utiliser ces variantes qui afficherons d'une meilleure manière. Mettez en commentaire à chaque fois quand vous utilisez une manière différente :

```
#[Route(path : "/", name : "home")]
public function index(Request $request): Response{
    //le var_dump classique en PHP
    var_dump($request);
    //le var_dump amélioré avec laquelle on peut rajouter si on veut un die derriere
    //pour arreter la suite du code
    dump($request);
    die;
    //ici c'est la meme chose que de faire un Dump puis un Die => dd()
    dd($request);

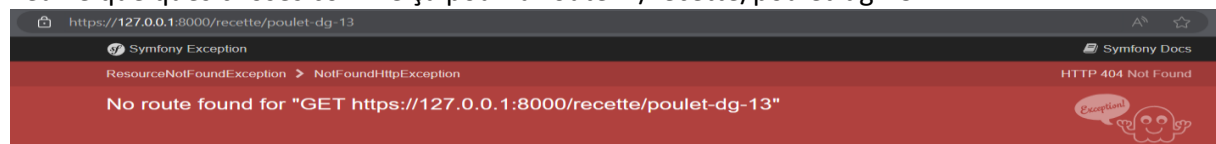
    return new Response("Hello ".$request->query->get('nom', 'World !!!'));
}
```

On voit bien dans notre navigateur qu'il a créé un objet avec des attributs à NULL. Avec le dump en enlevant « **die** » vous remarquerez qu'il continuera l'affichage de la page.

A noter que quand maintenant on fera un **dump(\$request)** sans le « **die** », les données se retrouveront dans la barre de debug de Symfony qui a disparu pour le moment. A vous de choisir ce que vous préférez.

3.2 Une autre page avec route particulière

Je vais maintenant essayer de créer une route particulière. Imaginons que dans mon URL je veuille quelques choses comme ça pour la route « /recette/poulet-dg-13 » :



Bien entendu Symfony me dit qu'il n'a pas trouvé cette route donc erreur. Mais on va essayer de faire en sorte que ça passe. Pour faire ça on va devoir tout d'abord créer un nouveau controller.

Exos

- 1) Essayez de créer un Controller RecipeController qui permettra de gérer des recettes de cuisines.
- 2) Elle aura une méthode « show », une route avec comme chemin « /recette » et comme nom de route « app_recipe_show ».
- 3) Je veux qu'elle m'affiche dans votre navigateur « Bienvenue sur la page des recettes !!! »

Bienvenue dans la page des recettes !!!

- 4) (Difficile) Essayez de faire en sorte que si je mets « ?recette=Pundu » il me met :

Bienvenue dans la page Pundu

On va un peu modifier notre méthode « **show** » dans le « **RecipeController** » pour qu'elle fonctionne avec « **/recette/poulet-dg-13** ». On va y ajouter des paramètres entre accolade dans le chemin, un Slug qui représentera le nom de la recette et un id qui sera le numéro de la recette.

```
RecipeController.php U X
src > Controller > RecipeController.php > RecipeController > show
1  <?php
2
3  namespace App\Controller;
4
5  use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6  use Symfony\Component\HttpFoundation\Request;
7  use Symfony\Component\HttpFoundation\Response;
8  use Symfony\Component\Routing\Attribute\Route;
9
10 class RecipeController extends AbstractController
11 {
12     #[Route('/recette/{slug}-{id}', name: 'app_recipe_show')]
13     public function show(Request $request): Response
14     {
15         return new Response("Bienvenue dans la page ".$request->query->get('recette', 'des
16         recettes !!!'));
17     }
18 }
```

Si j'actualise ma page en mettant bien l'URL avec « **/recette/poulet-dg-13** », ça va fonctionner. Si on fait un **dump** et **die** de **\$request**, on peut voir dans « **attributes** » puis « **parameters** » qu'on a plusieurs paramètres dont un slug et un id. Il y a un petit problème non ?

```

https://127.0.0.1:8000/recette/poulet-dg-13

RecipeController.php on line 15:
Symfony\Component\HttpFoundation\Request {#2 ▾
  +attributes: Symfony\Component\HttpFoundation\ParameterBag {#15 ▾
    #parameters: array:8 [▾
      "stopwatch_token" => "173a37"
      "_route" => "app_recipe_show"
      "_controller" => "App\Controller\RecipeController::show"
      "slug" => "poulet"
      "id" => "dg-13"
      "route_params" => array:2 [▾]
      "firewall_context" => "security.firewall.map.context.main"
      "_access_control_attributes" => null
    ]
  }
}
```

On peut voir que « **slug** » contient juste « **poulet** » et que « **id** » contient par contre « **dg-13** ». Nous on voulait que « **id** » contienne juste **13** bien entendu.

Déjà pour que ce soit plus lisible on va faire un DD sur ces 2 attributs qui nous intéressent. Faites ça : **dd(\$request->attributes->get('slug') , \$request->attributes->get('id'));**

Là on verra mieux notre problème. Par défaut quand on met slug en paramètre, il ne sait pas trop ce qu'il doit prendre, on va devoir rajouter des précisions grâce aux « **requirements** ». On va lui dire que pour « **id** » je ne veux que des nombres et que pour « **slug** » je peux avoir des lettres en minuscule, des nombres et des tirets. C'est avec des expressions régulières qu'on fera ça (Regex) :

```
#[Route('/recette/{slug}-{id}', name: 'app_recipe_show', requirements: ['id'=> '\d+', 'slug'=> '[a-z0-9-]+'])]
```

Si on veut que notre « **id** » soit en entier, on peut remplacer dans le dd au niveau de l'id, **get** par **getInt**.

Il faut savoir qu'on aurait pu simplifier encore plus l'écriture dans le dd. En mettant en paramètres dans la méthode **show**, une chaîne de caractère **\$slug** et un entier **\$id**. Attention, il faut que ce soit exactement les mêmes noms que dans votre chemin. Et ensuite les utiliser comme variable.

```
#[Route('/recette/{slug}-{id}', name: 'app_recipe_show', requirements: ['id'=> '\d+', 'slug'=> '[a-z0-9-]+'])]
public function show(Request $request, string $slug, int $id): Response
{
    // dd($request->attributes->get('slug') , $request->attributes->getInt('id'));
    dd($slug, $id);
    return new Response("Bienvenue dans la page ".$request->query->get('recette', 'des recettes !!!'));
}
```

Voilà on a fait un petit tour sur l'utilisation de base d'un Controller. Ce qu'il faut retenir c'est qu'avec Symfony lorsqu'on souhaite créer et gérer une nouvelle URL donc une nouvelle page, on va créer une nouvelle action/méthode dans un controller. On peut soit créer un nouveau controller, soit tout simplement rajouter une méthode/action à un controller existant.

La finalité c'est d'arriver à faire quelque chose comme ça :

```
RecipeController.php U x
src > Controller > RecipeController.php > ...
1  <?php
2
3  namespace App\Controller;
4
5  use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6  use Symfony\Component\HttpFoundation\Request;
7  use Symfony\Component\HttpFoundation\Response;
8  use Symfony\Component\Routing\Attribute\Route;
9
10 class RecipeController extends AbstractController
11 {
12
13     #[Route('/recette', name: 'app_recipe_index')]
14     public function index(Request $request): Response
15     {
16         return new Response("Bienvenue dans la page des recettes !!!");
17     }
18
19     #[Route('/recette/{slug}-{id}', name: 'app_recipe_show', requirements: ['id'=> '\d+', 'slug'=> '[a-z0-9-]+'])]
20     public function show(Request $request, string $slug, int $id): Response
21     {
22         // dd($slug, $id);
23         return new Response("Recette numéro ". $id . " : ". $slug);
24     }
25 }
```

Ce qui donnera 2 pages, une pour l'index et une qui montre la recette qu'on veut.

L'index :

← ↻ 🏠 🔒 https://127.0.0.1:8000/recette

Bienvenue dans la page des recettes !!!

Et le show :

← ↻ 🏠 🔒 https://127.0.0.1:8000/recette/poke-bowl-saumon-5

Recette numéro 5 : poke-bowl-saumon

Si vous voulez faire en sorte que nos données sortent sous forme de Json dans le show, on peut utiliser l'objet **JsonResponse** comme type de retour. Il suffit de changer le return par :

```
return new JsonResponse([
    'id' => $id,
    'slug' => $slug
]);
```

Attention, il doit importer JsonResponse de HttpFoundation.

← ↻ 🏠 🔒 https://127.0.0.1:8000/recette/poke-bowl-saumon-5

```
1  {
2      "id": 5,
3      "slug": "poke-bowl-saumon"
4  }
```

Ou encore plus simple juste en mettant \$this->json :

```
return $this->json([
    'id' => $id,
    'slug' => $slug
]);
```

Le format Json est très important car la plupart des Api l'utilise.

A noter que si vous voulez savoir toutes les routes de votre projet, il suffit de taper dans la console :

- **symfony console debug:router**

On y verra les routes liées au Framework, à la sécurité, au profiler et nos routes qu'on a créées dans HomeController et RecipeController. Pour l'instant tout est en ANY mais on y reviendra par après.